Week 7: Revision Notes

Page 1 — Cleaning and searching text

Python strings are immutable: methods such as lower(), strip(), and replace() produce a new string. Assign the result if you want to keep it.

Normalise incoming data before comparisons. For example, a server named " SRV-01 " and "srv-01" should match after strip().lower().

Whitespace matters when parsing. strip() removes surrounding spaces, while split() without an argument treats multiple spaces as separators.

Substring searching: use in for a Boolean; find() for a location, where -1 means missing. startswith() and endswith() are ideal for filtering log levels and filenames.

replace() changes occurrences of a substring; use caution with case-sensitive replacement. For case-insensitive filtering, compare with line.lower().

Practical security example: read a log line, test if "failed" in line.lower(), and display it for review.

Week 7: Revision Notes

Page 2 — Parsing, error handling and reports

Use split(",", 1) to separate the first field from the remaining fields without destroying the rest of the record. Use split("=", 1) for key-value pairs.

Example input: " SRC=10.0.0.5, ATTEMPTS=12, RESULT=FAILED ". Strip each piece; normalise keys and result to lowercase; convert attempts with int().

A reusable parse_line(raw_line) function should return a dictionary. A separate status_of(record) function decides whether to flag a record. Keep parsing and classification separate.

Try/except ValueError handles malformed numbers. Also consider missing keys and blank records. Do not silently turn bad data into valid data.

Read a file with open() or pathlib.Path.read_text().splitlines(); process records in a loop; count successes, flags and invalid records; print a summary.

Debugging: too many values to unpack often means the wrong separator or missing maxsplit. Calling strip() on a float raises AttributeError; clean the string before conversion.

Revision checklist: case, whitespace, in/find, replace, startswith/endswith/count, split/maxsplit, join, f-strings, dictionaries, functions, exceptions and file input.